

# AI-Powered Autonomous Smart Agriculture

*Swarm Intelligence and Reinforcement Learning for an  
Autonomous Pollination System -- Figure Guide*

---

Department of Computer Science and Engineering  
University of Dhaka  
CSE 4101 -- Artificial Intelligence Laboratory

## Abstract

This document presents and explains every figure produced by the autonomous pollination system. Global optimization (PSO and a Genetic Algorithm) assigns flowers to robots; reinforcement learning (Value Iteration and Q-learning) then navigates a grid-world MDP. Every figure has a plain-language 'how to read it' breakdown of its axes, curves, colours and symbols. Start with the Notation Key on the next page.

# How to Read These Figures -- Markers

---

## **Coloured dots**

Flowers. On the allocation maps the colour tells you WHICH robot is responsible for that flower.

## **Magenta star**

A target flower the robot must reach and pollinate (a 'goal' cell) on the navigation grids.

## **Black square**

An obstacle -- an impassable cell (tree, rock, water). The robot can never enter it.

## **Green / lime square**

The robot's START cell (its depot / parking spot).

## **'P' marker (cyan or lime)**

A charging station -- where the robot must return once every flower is done.

## **Red x**

An obstacle on the allocation maps.

## **Small grey arrows (up/down/left/right)**

The POLICY: the single best direction to move from that cell. A whole field of arrows is a complete strategy.

## **Coloured line with dots**

The robot's actual PATH (trajectory) as it moves from cell to cell.

# How to Read These Figures -- Symbols & Terms

---

## **Fitness**

A single 'score' blending all six allocation objectives into one number. It is a COST, so LOWER IS BETTER.

## **Coverage**

Fraction of flowers actually pollinated (100% = every flower served). Higher is better.

## **Total distance**

Sum of all robots' flight distances, in metres.

## **Energy**

Battery units used -- directly proportional to distance.

## **Completion time / makespan**

The moment the LAST (slowest) robot finishes; the fleet is only done when everyone is done.

## **Workload std. dev. (sigma)**

How UNEVEN the flower counts are across robots. 0 = every robot has the same number (perfectly balanced); large = some robots overloaded.

## **Utilization**

Fraction of robots given any work at all.

## **w, c1, c2 (PSO)**

The three weights in PSO's velocity rule: w = inertia (keep going the same way), c1 = cognitive (pull to my own best), c2 = social (pull to the swarm's best).

## **pop, cx, mut (GA)**

GA settings: population size, crossover rate (how often two parents are mixed), mutation rate (how often a gene is randomly changed).

## **U(s)**

The VALUE of grid cell s: the total future reward the robot can expect if it starts there and acts optimally. High near flowers, low far away.

## **gamma**

Discount factor (0..1): how much a reward LATER is worth vs NOW. Near 1 = far-sighted.

## **theta**

Convergence threshold: stop iterating once values change by less than this tiny amount.

## **epsilon**

Exploration rate in Q-learning: probability of trying a RANDOM move instead of the current best.

## **alpha**

Learning rate in Q-learning: how big a step each new experience takes when updating a value.

## **'smoothed' / 'rolling' / '100-ep average'**

A moving average that hides episode-to-episode noise so the underlying TREND is visible.

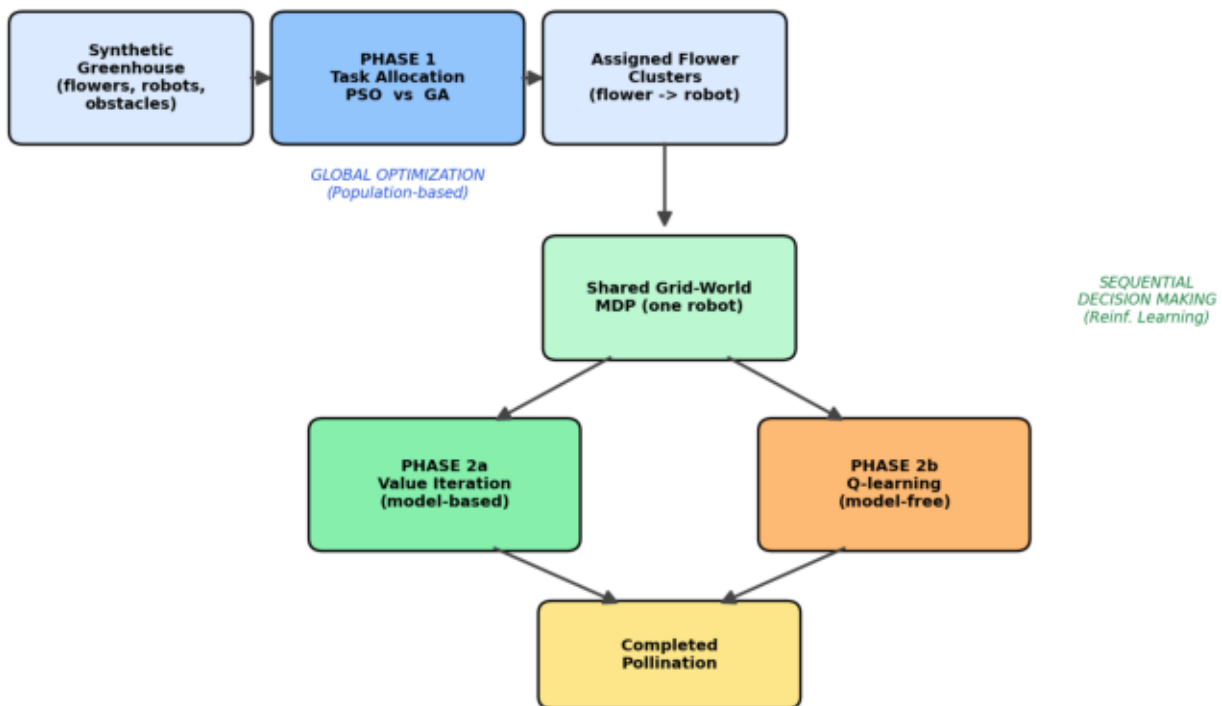
# 1. System Overview

---

Global optimization (PSO and a Genetic Algorithm) assigns flowers to robots; reinforcement learning (Value Iteration and Q-learning) then navigates a single robot through a grid-world MDP.

# End-to-end pipeline

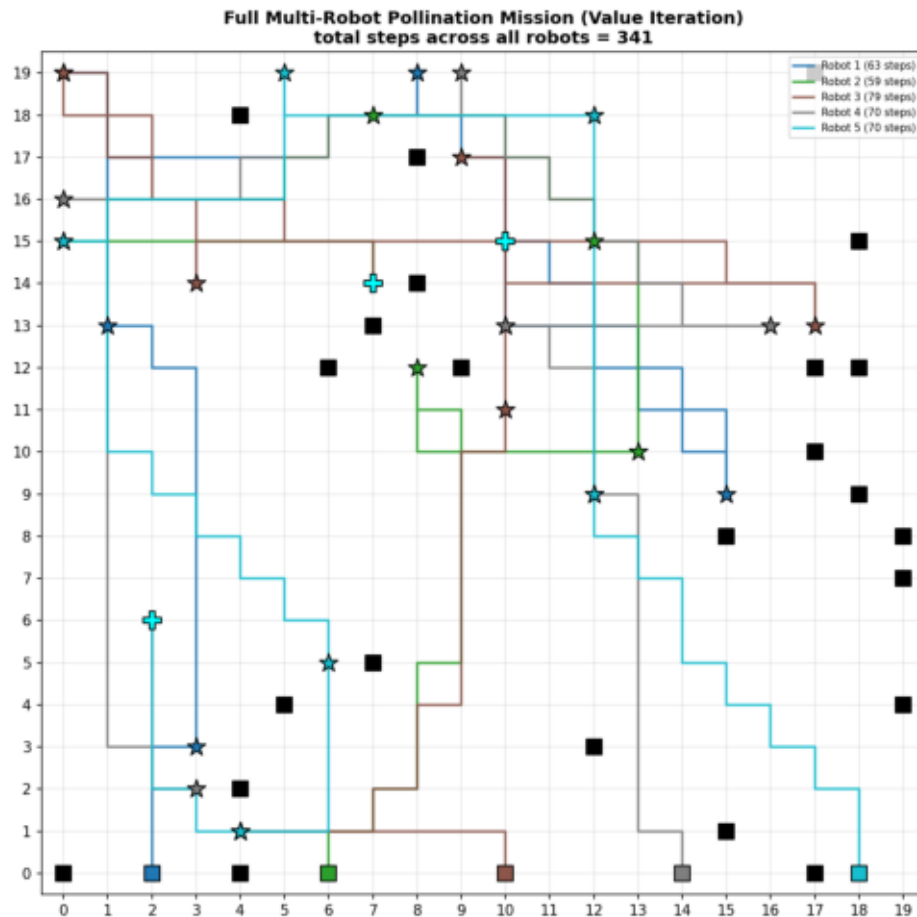
**Autonomous Pollination System -- End-to-End Pipeline**



## How to read this figure:

- Top row (blue boxes): Phase 1. The greenhouse feeds the allocator ('PSO vs GA'), which outputs flower clusters -- which robot owns which flowers.
- Middle (green box): those clusters define one shared grid-world MDP for a single robot to navigate.
- Two branches below: Phase 2 solves that grid two ways -- Value Iteration (left, model-based) and Q-learning (right, model-free).
- Bottom (yellow box): both branches re-join at 'Completed Pollination' -- either method finishes the mission.
- *Italic side captions name the paradigm of each half.*

# Full multi-robot mission



## How to read this figure:

- Axes: grid X and Y -- the greenhouse discretized into a 20x20 lattice of cells.
- Each colour = one robot. Its coloured line is its path; the coloured stars are its assigned flowers; the coloured square is where it starts.
- Shared black squares are obstacles; cyan 'P' are charging stations (common to all robots).
- Title reports total steps summed over the whole fleet -- evidence the COMPLETE system finishes, not just one robot.

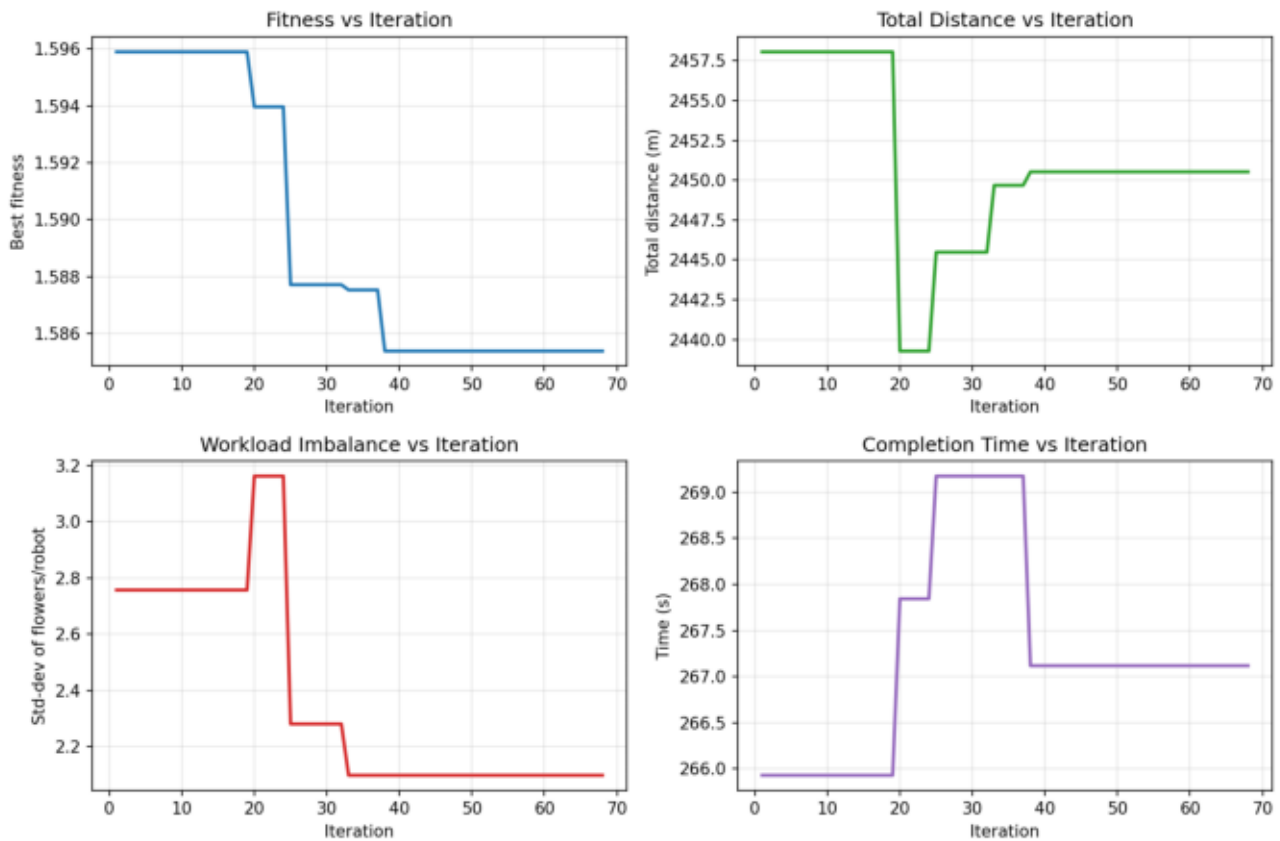
## 2. Phase 1 -- PSO Task Allocation

---

PSO encodes an assignment as a continuous vector (one dimension per flower, floored to a robot index) and minimizes the weighted multi-objective fitness from the notation key.

# PSO convergence behaviour

PSO Convergence Behaviour



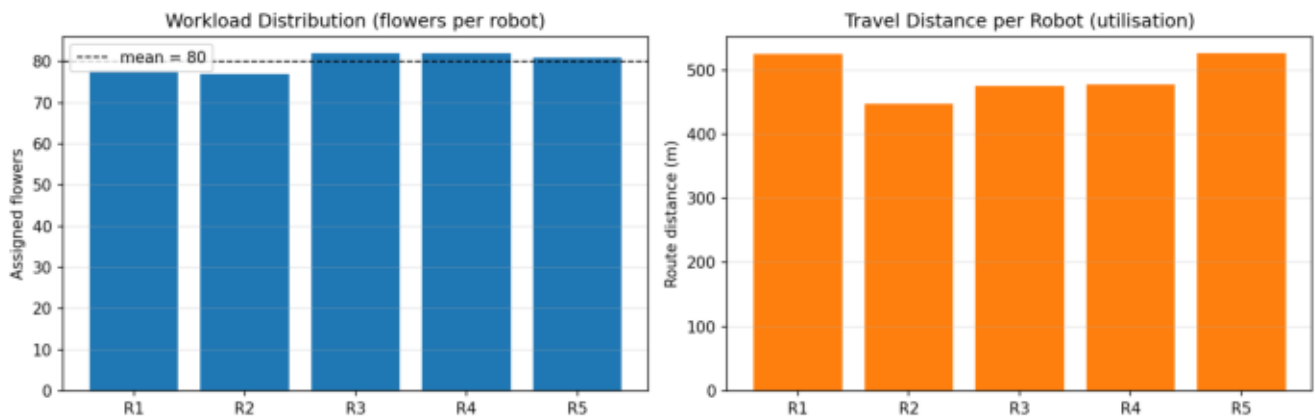
## How to read this figure:

- Four panels, all sharing the x-axis 'iteration' (one full swarm update). Every panel plots the BEST-SO-FAR value.
- Top-left, Fitness vs Iteration: the overall cost. Falls then flattens (improving, then converging). Lower = better.
- Top-right, Total Distance (m): total flight distance; may wobble since it is only one of six competing terms.
- Bottom-left, Workload Imbalance (sigma): how unevenly flowers are split. Falling = fairer.
- Bottom-right, Completion Time (s): the makespan (slowest robot's finish). Also trends down.
- Take-away: fitness dropping WHILE imbalance drops = PSO trades a little distance for a much fairer, cheaper plan.



## Workload and utilization

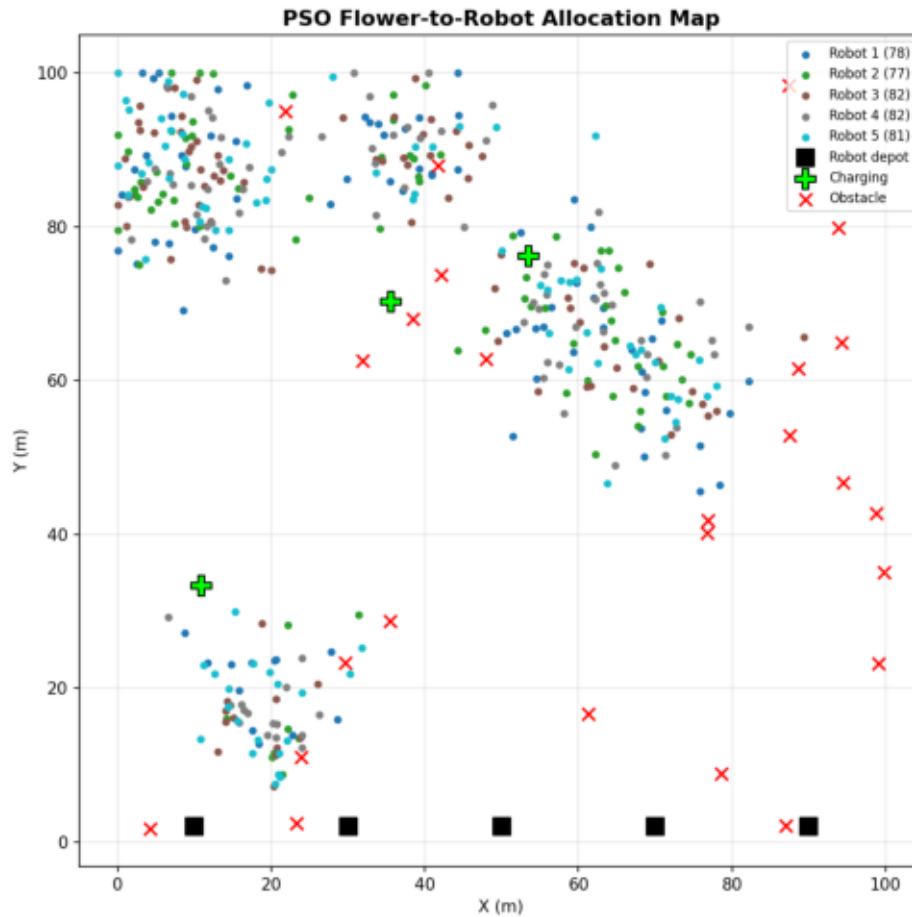
**Workload & Robot Utilisation (PSO)**



### How to read this figure:

- Two bar charts. 'R1'...'R5' on the x-axis are the five robots.
- Left, Workload Distribution: bar height = flowers assigned to each robot. Dashed line = the mean (~80). Bars near it = balanced.
- Right, Travel Distance per Robot: bar height = how far that robot flies. Similar heights = no robot overworked.
- Take-away: PSO gives every robot work and keeps counts AND distances even.

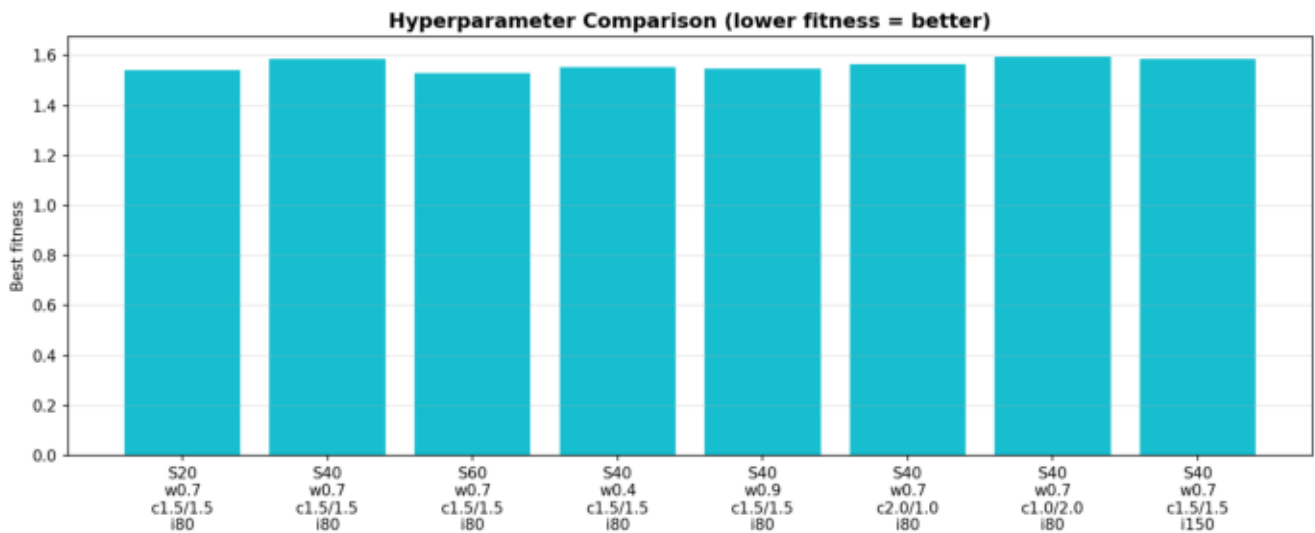
## PSO allocation map



### How to read this figure:

- Axes: X and Y in metres -- real positions in the 100x100 m greenhouse.
- Small dots = flowers, coloured by the robot assigned to them; legend gives each robot's flower count.
- Black squares = robot depots; lime 'P' = charging; red x = obstacles.
- Take-away: PSO's colours stay intermixed and it is only modestly better than random -- which motivates the GA next.

## PSO hyperparameter study



### How to read this figure:

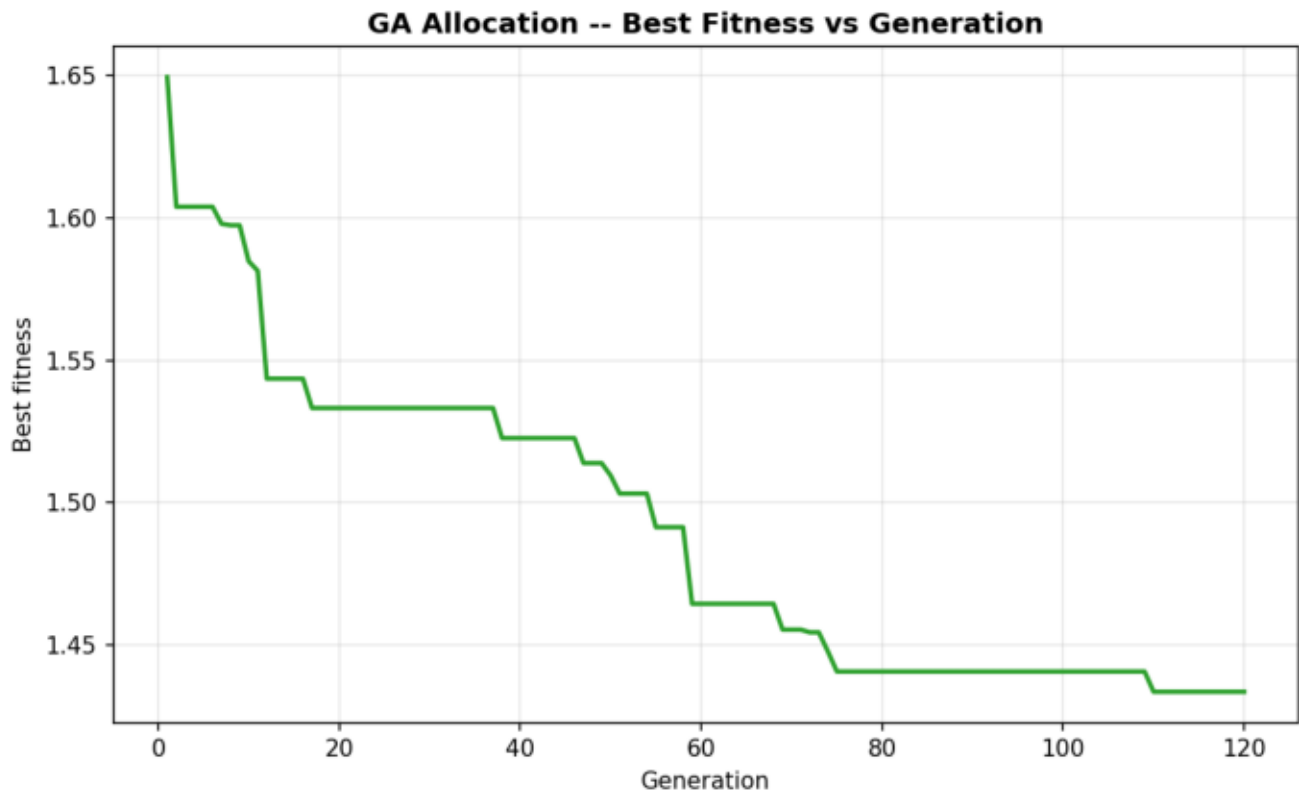
- Each bar = one PSO setting. The x-label encodes it: S=swarm size, w=inertia, c=c1/c2, i=iterations.
- Bar height = best fitness reached (lower is better).
- Take-away: flat-ish bars show PSO is fairly robust; the lowest bar justifies the default configuration used elsewhere.

### **3. Phase 1 -- GA Task Allocation (vs. PSO)**

---

A Genetic Algorithm solves the SAME problem with a DISCRETE encoding: a chromosome is 400 genes (one robot index per flower), evolved with tournament selection, uniform crossover, random-reset mutation and elitism, seeded with a distance-greedy assignment.

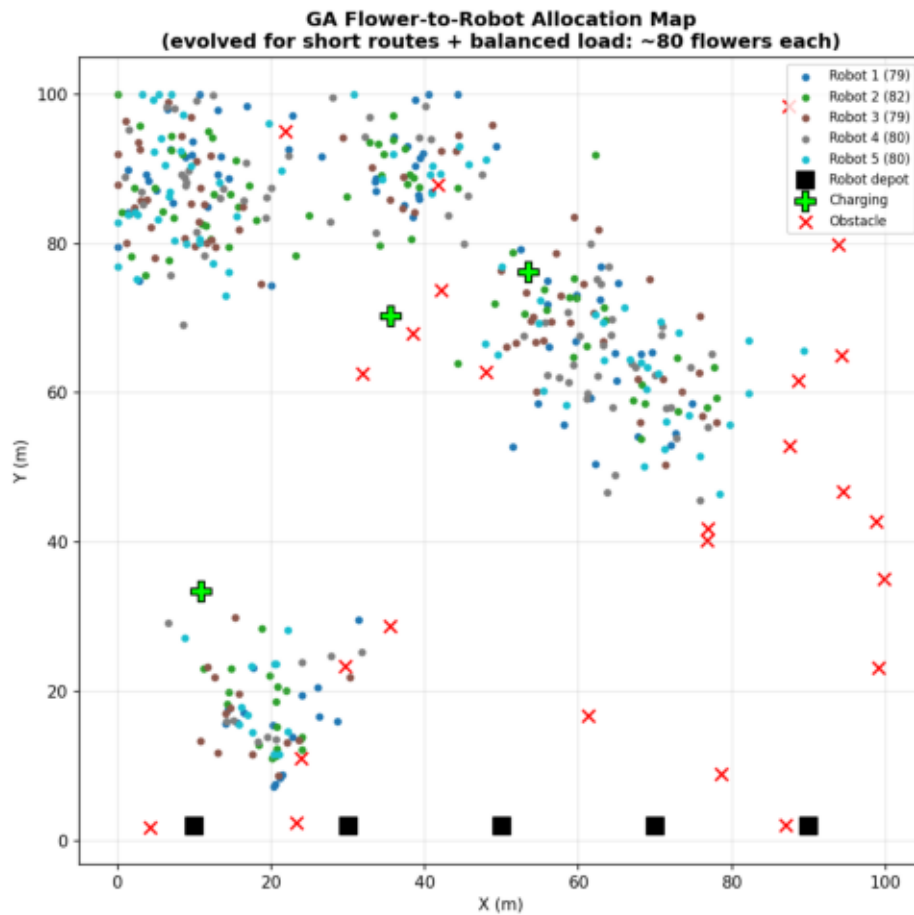
## GA allocation convergence



### How to read this figure:

- x-axis: generation (one cycle of select -> crossover -> mutate). y-axis: best fitness so far.
- The curve only ever goes DOWN because elitism always keeps the best chromosome -- the population can never get worse.
- Numbers: it falls from ~1.6 to ~1.43, landing BELOW PSO's ~1.59 -- the GA finds a better plan.

## GA allocation map



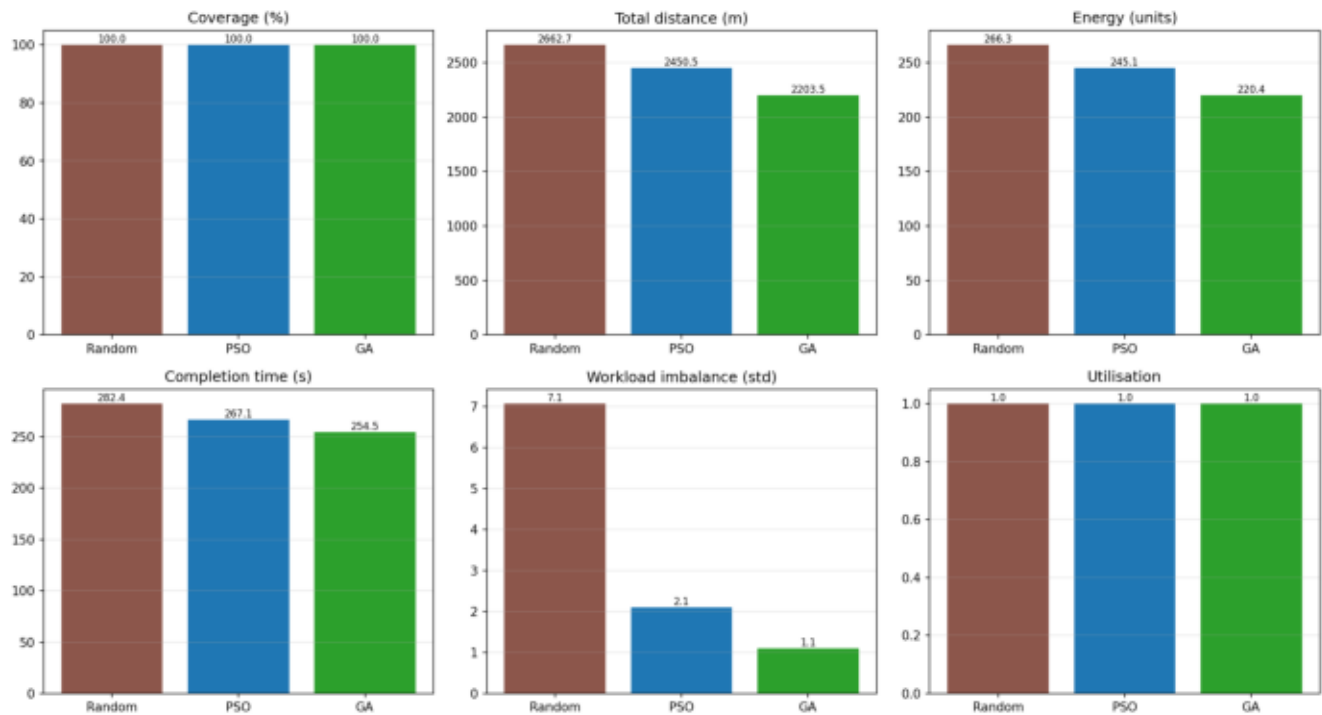
### How to read this figure:

- Read the markers exactly as the PSO map (dots = flowers by robot, black squares = depots, lime 'P' = charging, red x = obstacles).
- Legend counts: every robot gets 79-82 of the 400 flowers -- an almost perfectly even split.
- Honesty note: the colours look as intermixed as PSO's. Expected -- the 400 flowers sit in dense clusters that MUST be shared to balance loads. The GA's win is QUANTITATIVE (shorter routes, tighter balance, lower fitness), not a neater-looking map.

### 3. Phase 1 -- GA Task Allocation (vs. PSO)

## Random vs PSO vs GA

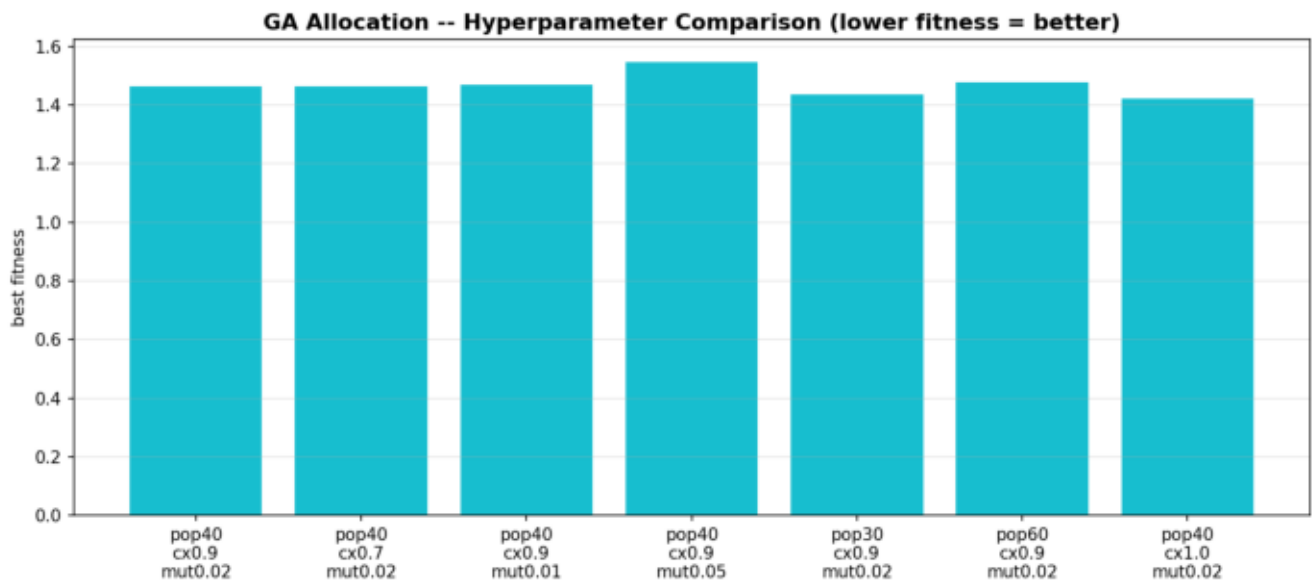
Task Allocation -- Random vs PSO vs GA



### How to read this figure:

- Six panels, one per objective. In each, the three bars are Random, PSO and GA; labels print the exact value.
- Coverage (%) and Utilisation: higher is better (all reach 100%).
- Total distance, Energy, Completion time, Workload sigma: LOWER is better. The GA (rightmost bar) is lowest in all four.
- Take-away -- the central Phase 1 result: the GA is the strongest allocator, beating PSO which beats Random.

## GA hyperparameter study



### How to read this figure:

- x-label encodes the setting: pop=population size, cx=crossover rate, mut=mutation rate.
- Bar height = best fitness (lower is better).
- Take-away: the tall (bad) bar at high mutation (0.05) shows too much random change destroys good chromosomes -- small mutation + strong crossover is the right balance.



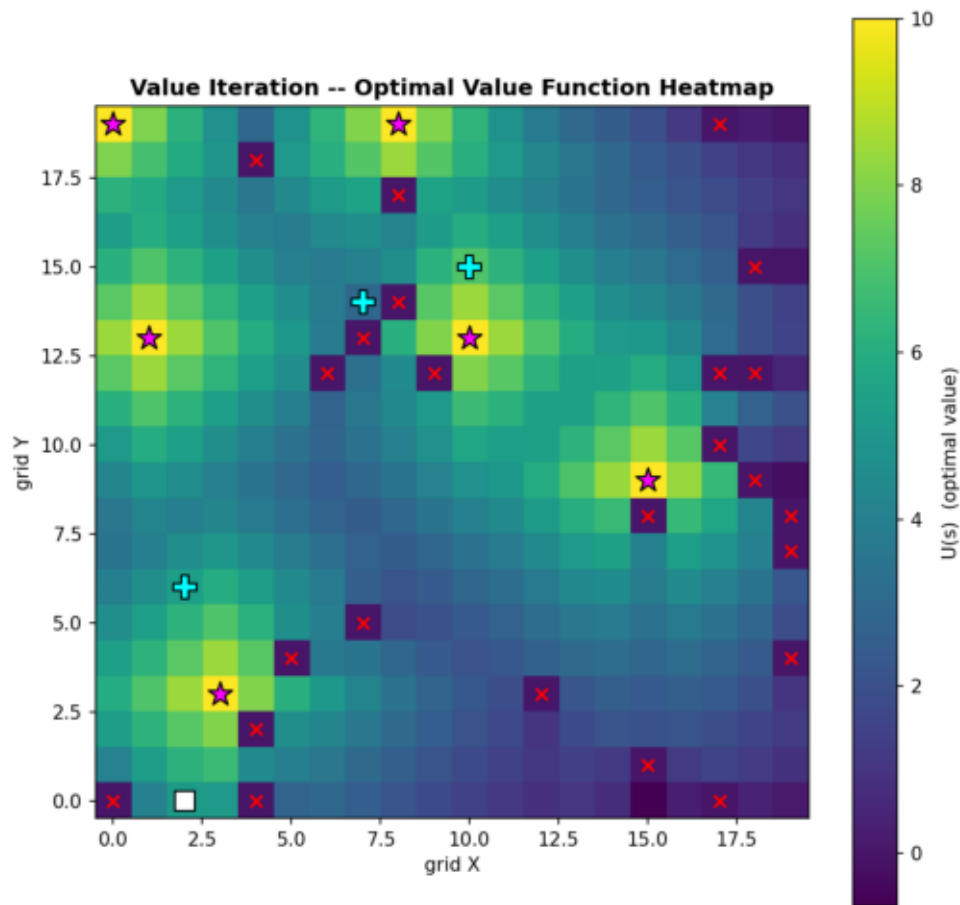
## 4. Phase 2 -- Value Iteration (model-based)

---

The environment is fully known. Value Iteration applies the Bellman update  $U(s) = \max_a \sum_{s'} T(s,a,s') [ R(s,a,s') + \gamma U(s') ]$  until convergence, then extracts the greedy policy.  $R$  is -0.1 for a normal step and -5 for a wall-blocked move, so VI feels the same collision penalty as Q-learning.

#### 4. Phase 2 -- Value Iteration (model-based)

### Optimal value function $U(s)$

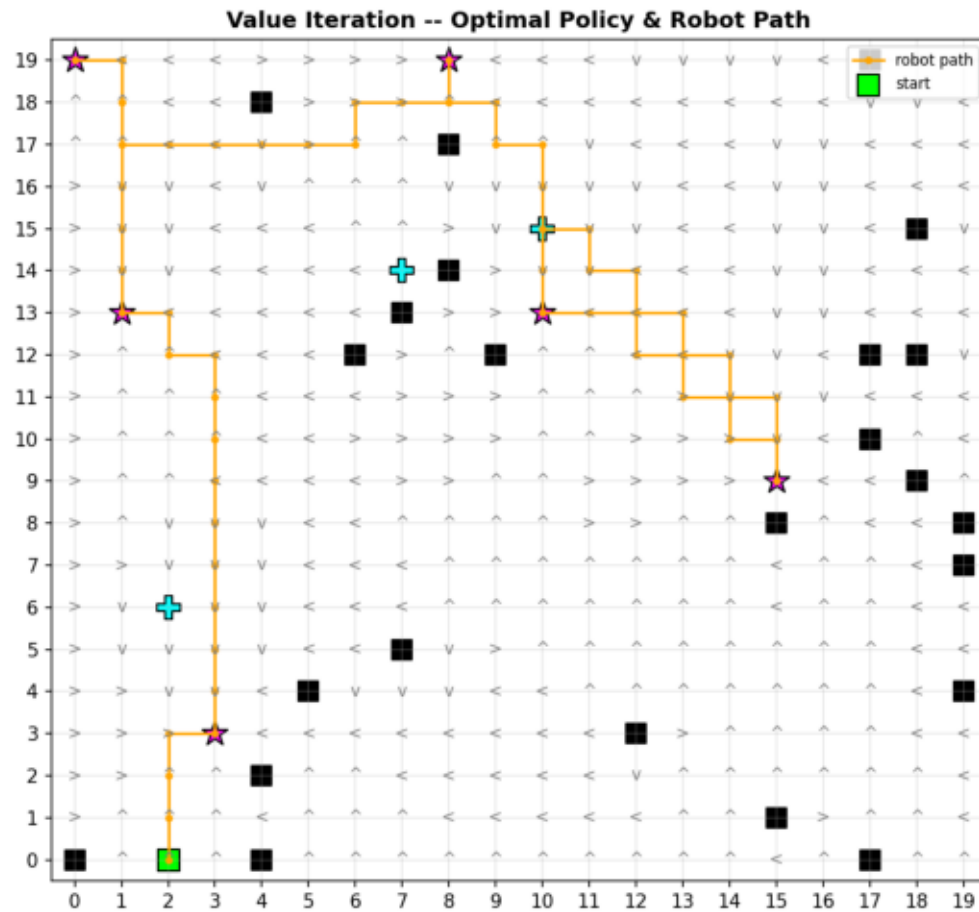


#### How to read this figure:

- Colour = value  $U(s)$  of each cell (see the colourbar): bright yellow = high value (good place), dark purple = low.
- Value is HIGHEST at the flowers and fades smoothly with distance -- like a hill with flowers at the peaks.
- Markers: magenta star = flower, red x = obstacle, cyan 'P' = charging, white square = start.
- Take-away: the robot only has to walk 'uphill' on this surface to reach a flower -- exactly what a correct value function looks like.

#### 4. Phase 2 -- Value Iteration (model-based)

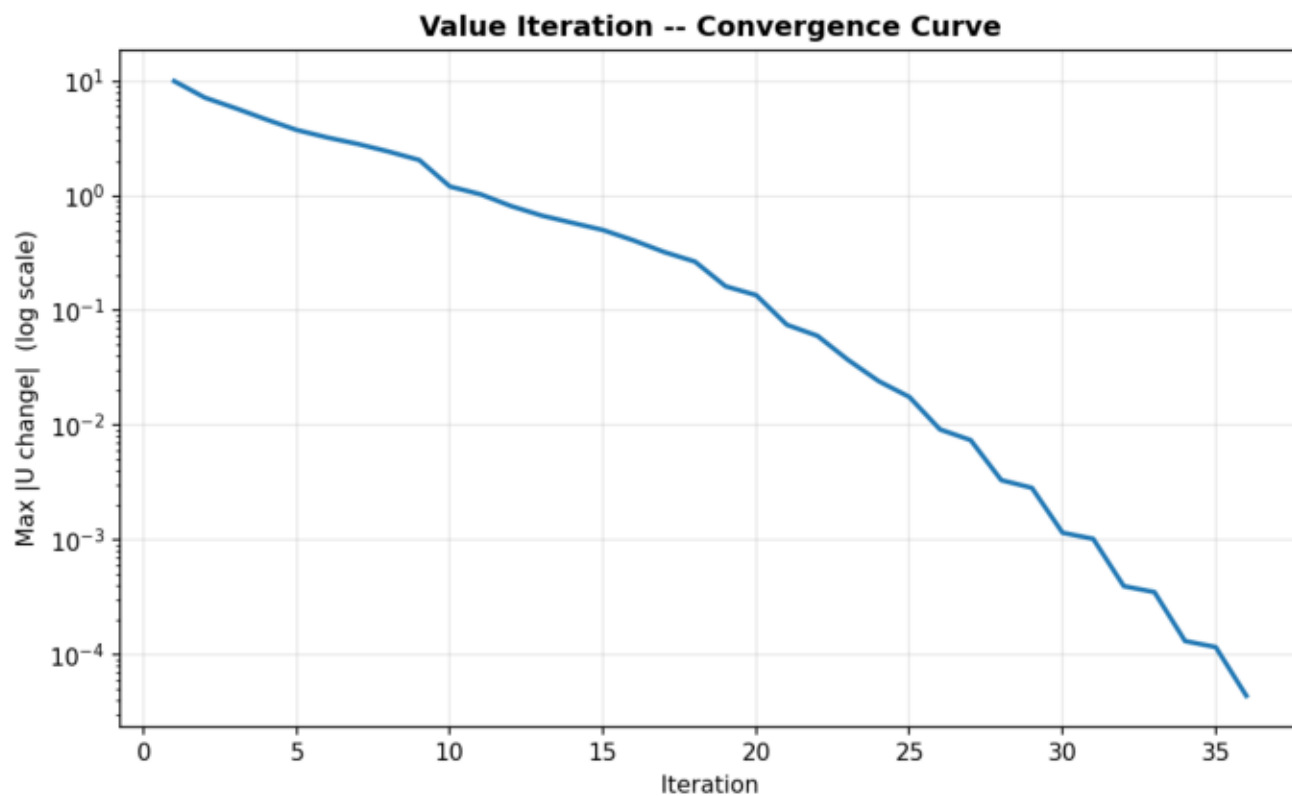
### Optimal policy and executed path



#### How to read this figure:

- Grey arrows = the policy (best action in every cell). Follow them from any cell and you reach a flower optimally.
- Orange line with dots = the robot's actual executed mission (visit all flowers, then return to charge).
- Markers: lime square = start, stars = flowers, cyan 'P' = charging, black squares = obstacles.
- Take-away: the orange path faithfully follows the grey arrows -- policy and value agree.

## Value Iteration convergence



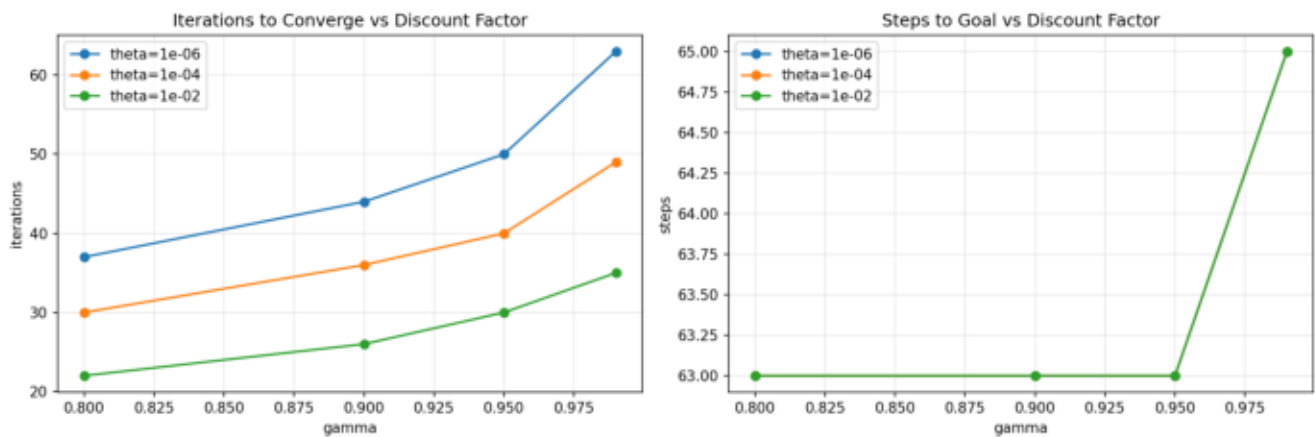
### How to read this figure:

- x-axis: iteration (one Bellman sweep over all cells).
- y-axis (LOG scale): the biggest change in any cell's value that sweep. As it shrinks, values are settling.
- A straight line on a log axis = error shrinks by a constant factor each step = geometric (fast, guaranteed) convergence.
- Take-away: it crosses the stop threshold  $\theta$  in about 36 iterations.

#### 4. Phase 2 -- Value Iteration (model-based)

### Value Iteration hyperparameter study

Value Iteration -- Hyperparameter Study

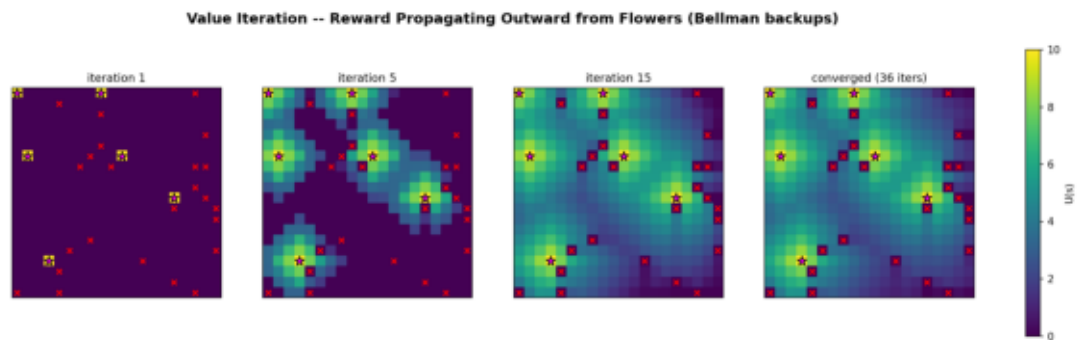


#### How to read this figure:

- Two panels vs the discount gamma (x-axis); one line per threshold theta.
- Left, Iterations to Converge: bigger gamma spreads value further so it needs MORE sweeps; smaller theta (stricter) also needs more.
- Right, Steps to Goal: the planned path length -- stays essentially FLAT at ~63 steps.
- Take-away: these settings change only the COMPUTE COST (left), not the QUALITY of the final path (right).

#### 4. Phase 2 -- Value Iteration (model-based)

### Value propagation (Bellman backups)



#### How to read this figure:

- Four heat-map snapshots, left to right, after 1, 5, 15 and (converged) iterations; same colour scale ( $U$  from 0 to 10).
- Panel 1: only cells TOUCHING a flower are bright -- value spread one ring.
- Panels 2-3: the bright region grows outward, ring by ring.
- Panel 4 (converged): the whole reachable grid has a sensible value pointing back to the flowers.
- Take-away: this literally shows the Bellman update 'backing up' reward one step per sweep.

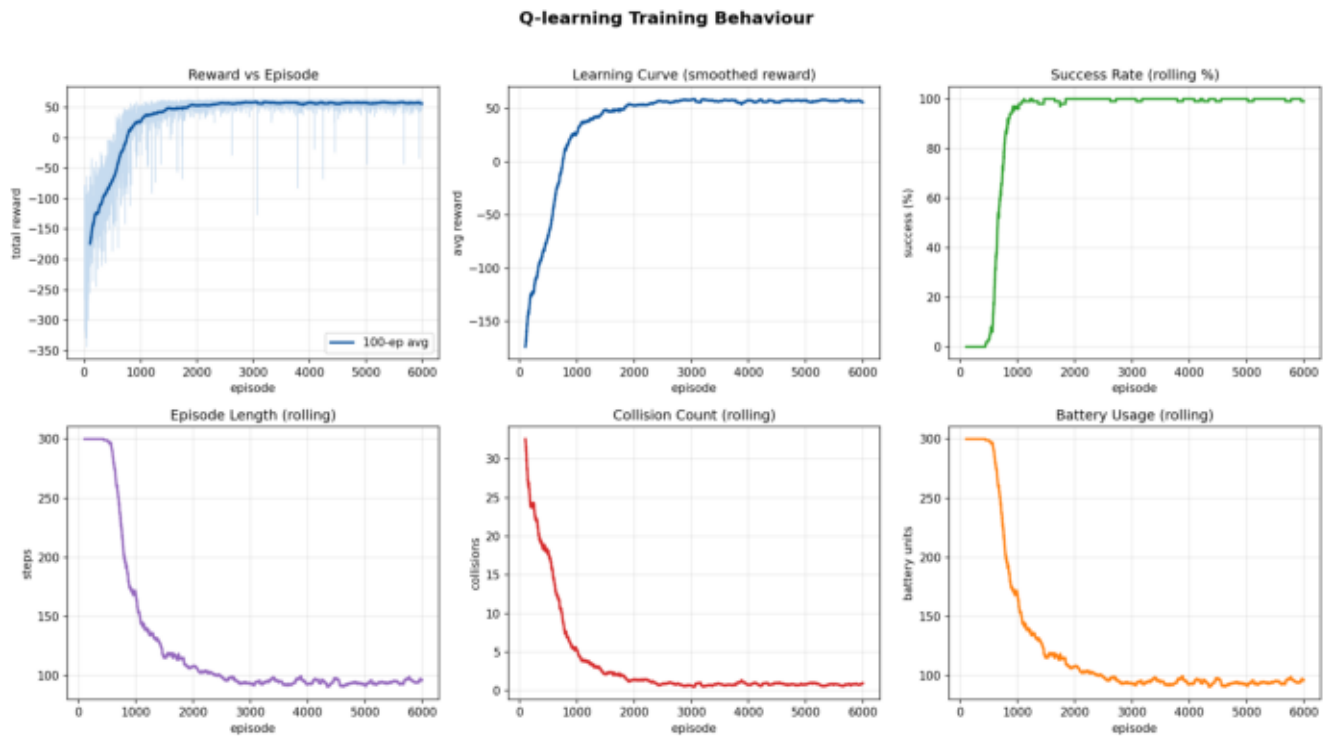
## 5. Phase 2 -- Q-learning (model-free)

---

The environment is treated as unknown. The agent learns by trial and error with an epsilon-greedy policy and the update  $Q(s,a) \leftarrow Q(s,a) + \alpha [r + \gamma \max_{a'} Q(s',a') - Q(s,a)]$ . The state carries a bitmask of already-pollinated flowers so the memoryless agent can finish the multi-flower mission.

## 5. Phase 2 -- Q-learning (model-free)

### Q-learning training behaviour

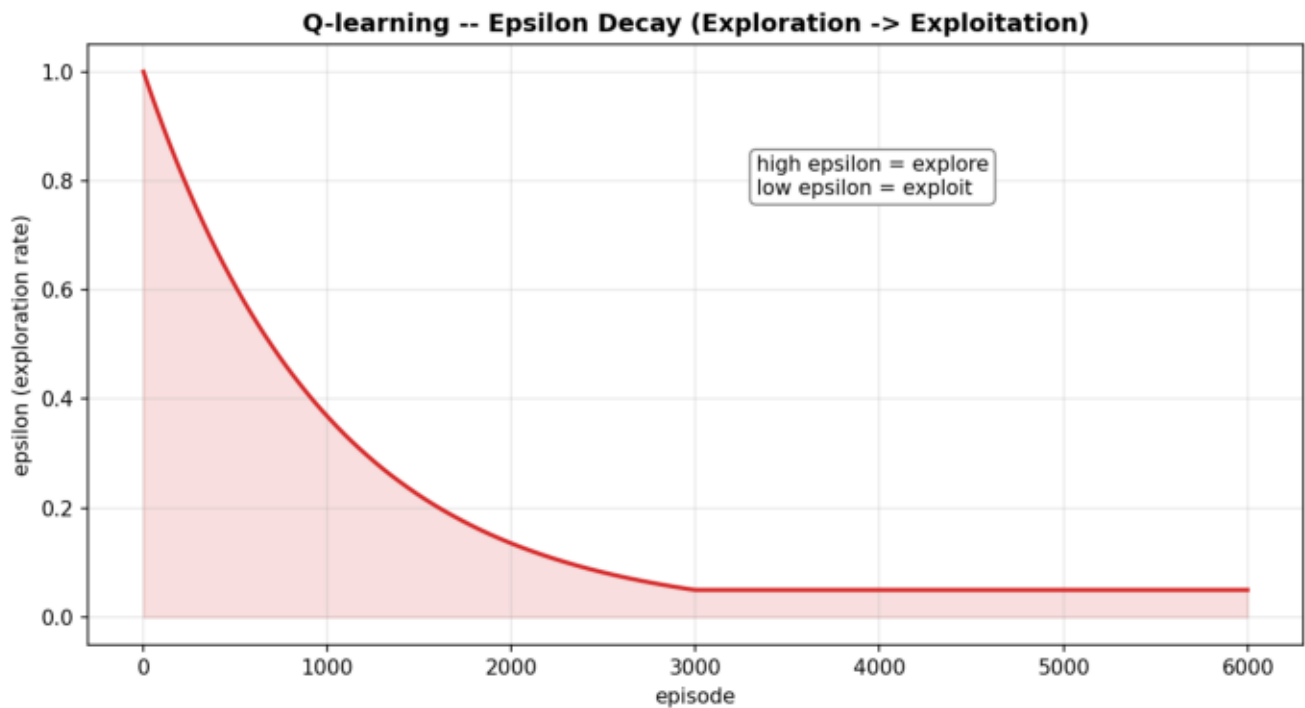


#### How to read this figure:

- Six panels, all with x-axis 'episode' (one full attempt at the mission).
- Top-left, Reward vs Episode: pale line = raw noisy reward; dark line = 100-episode average. Climbs from negative to positive.
- Top-middle, Learning Curve: just the smoothed reward, so the trend is crisp.
- Top-right, Success Rate (%): rolling share of episodes that finish the mission; climbs to 100%.
- Bottom-left, Episode Length: rolling steps per episode -- FALLS as the agent stops wandering.
- Bottom-middle, Collisions: rolling wall-bumps -- FALLS as it learns to avoid obstacles.
- Bottom-right, Battery Usage: rolling energy -- falls with shorter routes. Take-away: reward/success up, length/collisions/battery down = successful learning.



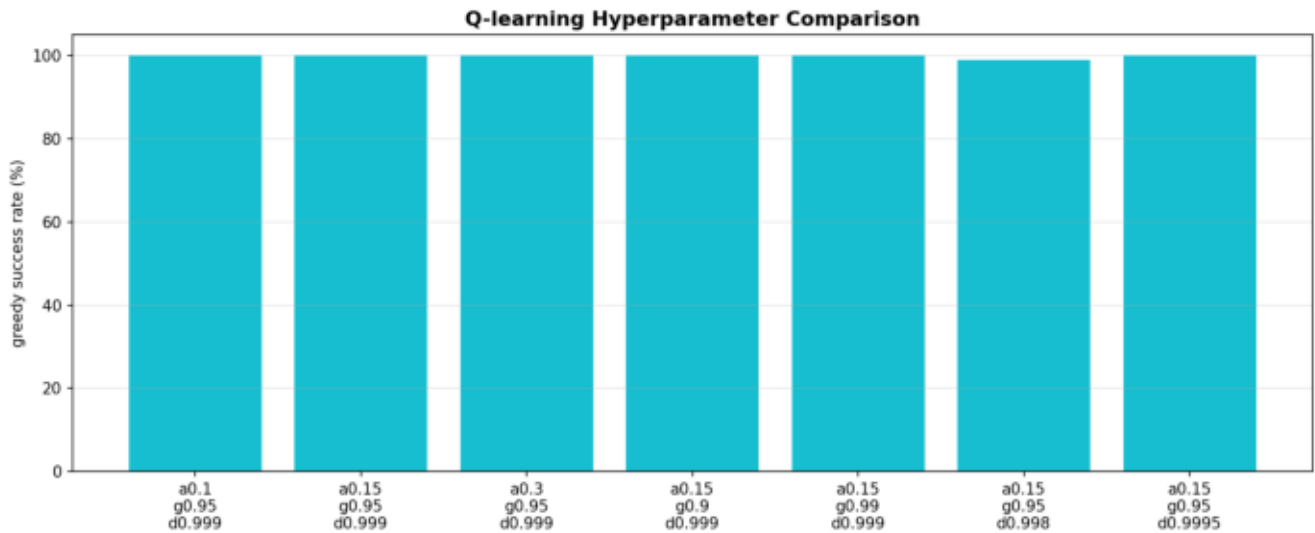
## Epsilon decay schedule



### How to read this figure:

- x-axis: episode. y-axis: epsilon, the chance of a RANDOM move.
- Decays from 1.0 (start: 100% random -- pure exploration, the agent knows nothing) to a floor of 0.05 (mostly exploiting).
- The annotation spells it out: high epsilon = explore, low epsilon = exploit.
- Take-away: this schedule is WHY the reward curve can rise -- early randomness discovers good moves, later greediness locks them in.

## Q-learning hyperparameter study



### How to read this figure:

- x-label encodes the setting: a=alpha (learning rate), g=gamma (discount), d=epsilon-decay.
- Bar height = greedy success rate (%); here higher is better.
- Take-away: most settings reach 100%, but a too-FAST decay (stops exploring too early) visibly lowers its bar.

## 6. Value Iteration vs. Q-learning

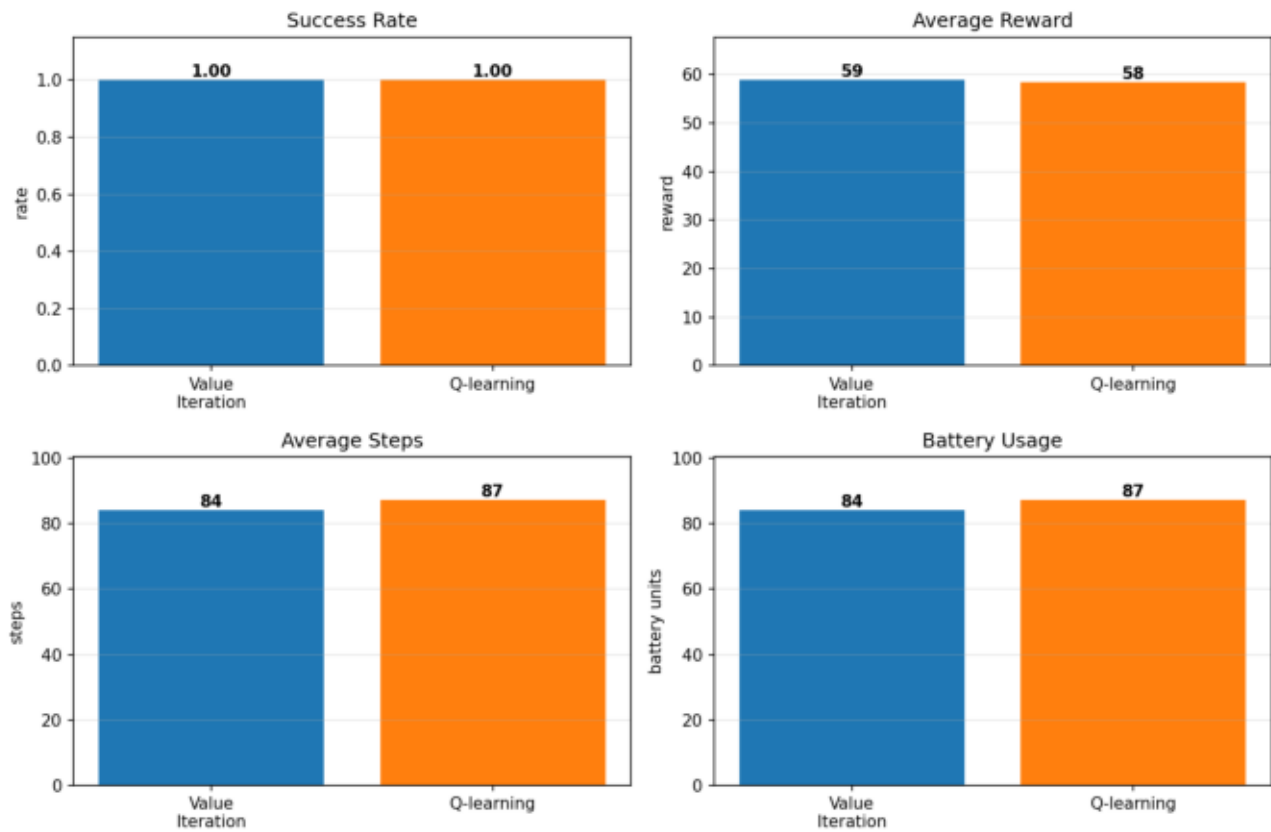
---

Both methods solve the identical grid and are scored the same way -- rolled out over 200 episodes WITH the 0.8/0.1/0.1 actuator noise on -- so the comparison is strictly like-for-like.

## 6. Value Iteration vs. Q-learning

### VI vs Q-learning performance

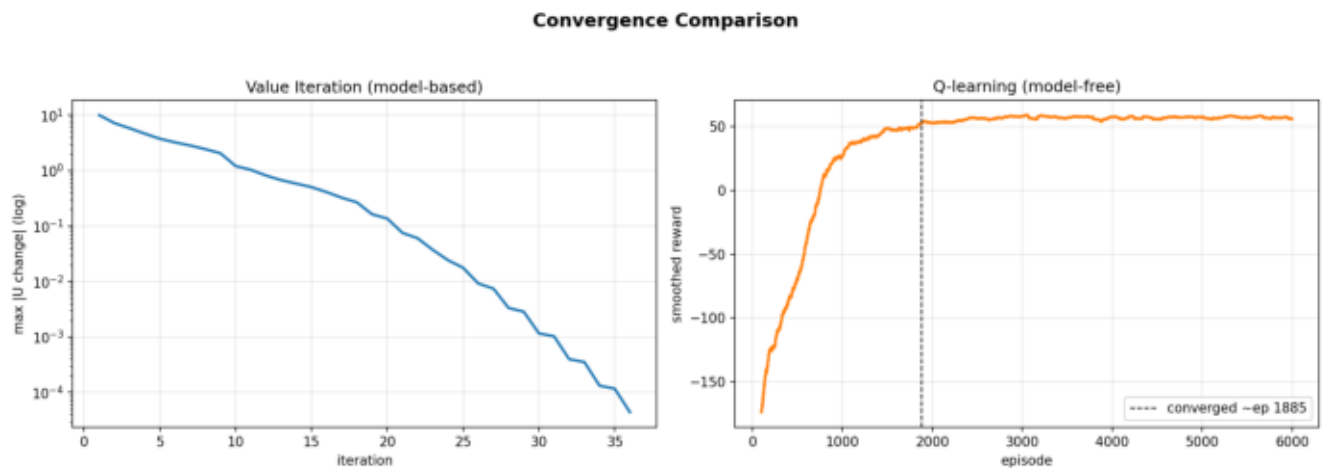
Value Iteration vs Q-learning -- Performance Comparison



#### How to read this figure:

- Four panels; in each, the BLUE bar is Value Iteration and the ORANGE bar is Q-learning. Value labels sit on top of each bar.
- Success Rate: both 100%. Average Reward: VI slightly higher (better). Average Steps & Battery: VI slightly lower (better).
- Convergence is deliberately NOT a bar here -- VI counts Bellman iterations, QL counts episodes (different units); the like-for-like view is the next figure.
- Take-away: the classic model-based vs model-free trade-off -- VI is a touch better and far faster BECAUSE it already knows the map.

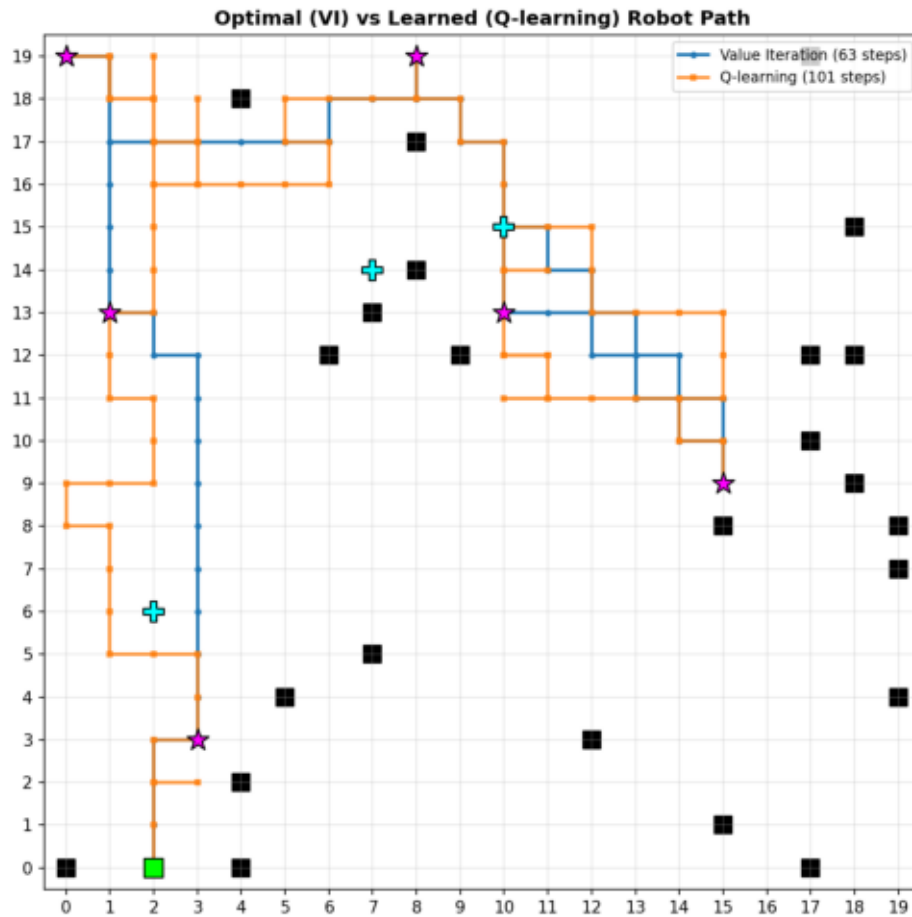
## Convergence comparison (like-for-like)



### How to read this figure:

- Two panels with DIFFERENT x-units, shown separately on purpose.
- Left, Value Iteration: max|U change| (log axis) vs iteration; plunges to the threshold in tens of iterations.
- Right, Q-learning: smoothed reward vs episode; the dashed vertical line marks convergence (~1900 episodes).
- Take-away: knowing the model (left) reaches the answer in a fraction of the effort that learning it (right) requires.

## Optimal vs learned path

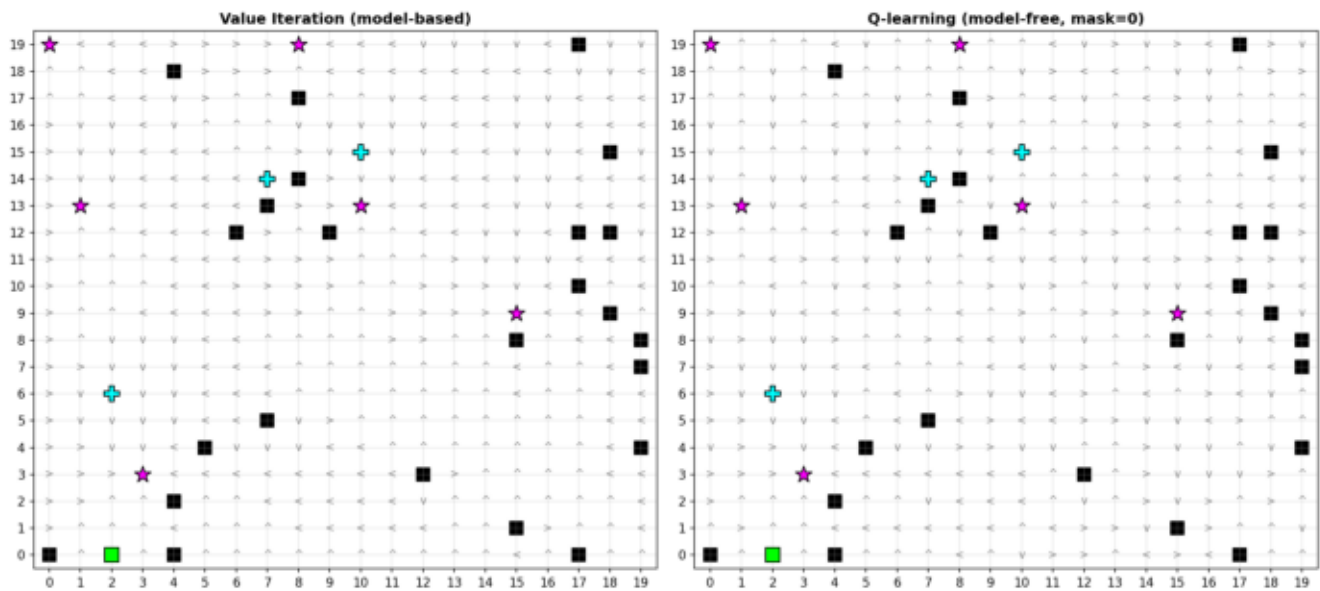


### How to read this figure:

- Blue line with circles = the VI (optimal) path; orange line with squares = the Q-learning (learned) path. Legend gives each step count.
- Grid markers as usual: stars = flowers, black squares = obstacles, cyan 'P' = charging.
- Take-away: both visit every flower and return to charge; the blue path is a little shorter -- you can SEE the cost of learning without a model.

## Optimal vs learned policy

Optimal Policy (Value Iteration) vs Learned Policy (Q-learning)



### How to read this figure:

- Two arrow maps side by side. Left = the VI policy. Right = the Q-learning policy for the 'no flower collected yet' state (mask=0).
- In both, a grey arrow in each cell is the best move; stars/squares/'P' mark flowers/obstacles/charging.
- Take-away: near the flowers the two arrow-fields point the same way -- model-free Q-learning REDISCOVERED essentially the optimal policy VI computed directly.

## 7. Summary of Key Results

**Table 1. Task allocation: Random vs. PSO vs. GA (identical greenhouse and fitness). Lower is better**

| Metric                    | Random | PSO  | GA          |
|---------------------------|--------|------|-------------|
| Fitness (lower is better) | 1.77   | 1.59 | <b>1.43</b> |
| Total distance (m)        | 2663   | 2451 | <b>2204</b> |
| Energy (units)            | 266    | 245  | <b>220</b>  |
| Completion time (s)       | 282    | 267  | <b>255</b>  |
| Workload std. dev.        | 7.1    | 2.1  | <b>1.1</b>  |
| Coverage / Utilization    | 100%   | 100% | <b>100%</b> |

**Table 2. Navigation: Value Iteration vs. Q-learning (same grid-world MDP, identical stochastic test)**

| Metric              | Value Iteration | Q-learning    |
|---------------------|-----------------|---------------|
| Success rate        | 100%            | 100%          |
| Average reward      | 58.8            | 58.4          |
| Average steps       | 84              | 87            |
| Battery usage       | 84              | 87            |
| Execution time (s)* | ~0.05           | ~9-17         |
| Convergence         | 36 iterations   | 1885 episodes |

\* Wall-clock time; hardware dependent and the only non-reproducible metric. Reward/steps/battery are averaged over 200 test episodes under the identical 0.8/0.1/0.1 actuator-noise model for BOTH methods, so the comparison is strictly like-for-like and reproducible under the fixed seed.